Arogya Bhoomi

System Architecture & Workflow Document

SIH 2026 · Problem Statement 193

Traceable Recovery & Deployment of Micronutrient Compounds from Expired Pharmaceuticals

Generated: *09 September 2026, 22:28*

Technology	Python 3.10 · Flask · SQLite · ReportLab · Leaflet.js
Test Suite	88 automated tests — 100% pass rate
API Endpoints	24 REST endpoints
Database Tables	8 SQLite tables
Compounds	Ferrous Sulphate (Fe) · Zinc Sulphate (Zn) · Potassium Chloride (K)
Deficiency Regions	29 mapped Indian states
Demo Intake Records	45 illustrative pharmaceutical batches

Table of Contents

1. High-Level System Architecture

The platform follows a **4-layer architecture** separating concerns across Client, Presentation, Application, and Data layers. All layers communicate through the central Flask application controller (**app.py**), which orchestrates API routing, business module delegation, audit logging, and PDF generation.

Layer	Components	Responsibility
■■ **Client Layer**	Web Browser	User interaction via HTTP requests and rendered HTML pages
■ **Presentation Layer**	Jinja2 Templates (base.html, index.html, stock.html, pipeline.html, dosage.html, soil_map.html, about.html)	Server-side HTML rendering, layout inheritance, sidebar navigation, dynamic content injection
■■ **Application Layer**	Flask app.py · Route Dispatcher · 24 REST API Endpoints · PDF Generator (ReportLab) · Audit Engine (SHA-256) · Startup Seeder	HTTP routing, request handling, response formatting, business logic orchestration, PDF generation, audit chain management
■ **Business Logic Layer**	Python Modules · intake.py — Compliance · geolocation.py — Location · matching.py — Soil Match · dosage.py — Dosage Calc	Encapsulated domain logic: whitelist validation, coordinate resolution, Haversine distance matching, severity-adjusted dosage calculation
■■ **Data Layer**	SQLite (app.db) — 8 tables JSON/CSV Reference Datasets	Persistent storage for batches, matches, partners, audit events, retests. Static reference data for compounds, soil deficiency, dosage rates

Data Flow Summary:

→ Browser → Flask Route Dispatcher → Jinja2 Template → Rendered HTML → Browser

→ Browser → Flask API Endpoint → Business Module → SQLite → JSON Response → Browser

→ Flask Startup → init_db_schema() → seed_demo_partners() → seed_demo_matches_and_audit() → recompute_and_fix_audit_hashes()

→ load_data.py → JSON/CSV Reference Files → SQLite (compound_reference, soil_deficiency, dosage_rates, intake_batches)

2. Database Schema (8 Tables)

The SQLite database (**app.db**) contains 8 tables. Four are seeded from reference datasets via **load_data.py**, and four are created at runtime by **app.py**'s **init_db_schema()**.

Table	Origin	Size	Purpose
compound_reference	Seeded	3 rows	Approved single-compound whitelist (FeSO■, ZnSO■, KCl). Used by intake.py for exact-match compliance validation.
soil_deficiency	Seeded	29 rows	State-level soil deficiency records. Each row contains deficient nutrients, usable compounds, and usable/not-usable status.
dosage_rates	Seeded	3 rows	Base application rates, severity multipliers, foliar specs. Raw JSON stores the full ICAR dosage record per compound.
intake_batches	Seeded	45 rows	Pharmaceutical batch intake ledger. Tracks batch ID, compound, quantity, source location, expiry, and compliance status.
matches	Runtime	Dynamic	Soil-deficiency match records created by pipeline execution. Links batch → target state, dosage, partner assignment, and handoff.
partners	Runtime	6 rows	Certified recovery partner facilities. Tracks name, type, location, supported compounds, capacity, and availability.
audit_events	Runtime	Dynamic	SHA-256 hash-linked append-only audit events. Each event references the previous event's hash (or 'GENESIS' for the first).
soil_retests	Runtime	Dynamic	Post-deployment soil re-test monitoring records. Tracks before/after values, improvement, and feedback.

Key Relationships:

- compound_reference ■validates→ intake_batches (compound_name exact match)
- compound_reference ■defines rates→ dosage_rates (compound_name lookup)
- intake_batches ■produces→ matches (batch_doc_id → batch_id)
- intake_batches ■tracks→ audit_events (batch_doc_id → batch_id)
- matches ■assigned to→ partners (assigned_partner_id → partner_id)
- matches ■monitors→ soil_retests (match_id → match_id)
- soil_deficiency ■target region→ matches (state → target_state)

3. Recovery Pipeline — 4-Stage Execution Flow

The core of the platform is the **4-stage automated recovery pipeline**, triggered by `POST /api/process_batch/<batch_id>`. The pipeline is **idempotent** — re-running the same batch does not create duplicate records.

Stage	Module / Function	Execution Steps	Gate / Notes
STAGE 1 Intake & Compliance	intake.py check_compliance()	1. Fetch batch from intake_batches 2. Exact-match compound_name against compound_reference 3. Set compliance_status = 'Accepted' or 'Rejected' 4. Audit: BATCH_LOGGED + COMPLIANCE_PASSED/REJECTED	Pipeline STOPS if status = 'Rejected'. Rejected batches cannot proceed.
STAGE 2 Geolocation Resolution	geolocation.py resolve_location()	1. Parse source_location ('City, State' or 6-digit pincode) 2. If pincode → resolve via indiapins package 3. If city → lookup in hardcoded city coordinate table 4. Fallback → state centroid coordinates 5. Output: latitude, longitude, district, state	Resolution always succeeds for demo data cities. State centroid fallback guarantees output.
STAGE 3 Soil-Deficiency Matching	matching.py find_match()	1. Check if source_state is 'usable' AND has compound → Same-State Match 2. If not, scan all 29 states for compound compatibility 3. Compute Haversine distance to each candidate state 4. Select nearest candidate → Nearest Fallback Match 5. Audit: MATCH_CREATED event recorded	Pipeline STOPS if no eligible state is found for the compound.
STAGE 4 Dosage Recommendation	dosage.py get_dosage()	1. Lookup compound in dosage_rates table 2. If base_rate = NULL → Foliar spray path (FeSO■) Return foliar_spec string directly 3. If base_rate exists → Soil application path adjusted_rate = base_rate × severity_multiplier 4. Audit: DOSAGE_GENERATED event recorded	FeSO■ NEVER uses kg/ha calculation. Always returns foliar spray spec.

Pipeline Flow Summary:

Batch Selected → [Stage 1: Compliance] → Accepted? → [Stage 2: Geolocation] → Coordinates Resolved → [Stage 3: Matching] → Region Found? → [Stage 4: Dosage] → Match + Dosage Saved to DB → Audit Events Chained → ■ Pipeline Complete → ■ Download PDF

Post-Pipeline Persistence:

1. INSERT match record into matches table (origin = 'pipeline')
2. Append BATCH_LOGGED audit event (prev_hash = 'GENESIS')
3. Append COMPLIANCE_PASSED audit event (prev_hash = hash of event 1)
4. Append MATCH_CREATED audit event (prev_hash = hash of event 2)
5. Append DOSAGE_GENERATED audit event (prev_hash = hash of event 3)
6. Each audit event's hash = SHA256(event_id | batch_id | type | timestamp | description | prev_hash)

4. Module Dependency Map

The application follows a **controller** → **module** → **database** pattern. Business logic is encapsulated in four independent Python modules under **modules/**. The Flask controller (**app.py**) orchestrates module calls and manages cross-cutting concerns (audit logging, PDF generation, startup seeding).

Component	File	Key Functions	Dependencies
Flask Controller	app.py (1981 lines)	process_batch() record_audit_event() verify_audit_chain() get_report_data_for_batch() generate_acknowledgement_pdf_bytes() init_db_schema() seed_demo_partners() seed_demo_matches_and_audit()	intake.py geolocation.py matching.py dosage.py SQLite ReportLab
Compliance Engine	modules/intake.py (106 lines)	check_compliance(batch_doc_id) run_all_pending()	SQLite (compound_reference, intake_batches)
Geolocation Resolver	modules/geolocation. py (223 lines)	resolve_location(location_string) distance_km(loc1, loc2) _parse_location_string() _resolve_via_pincode() _resolve_via_city_lookup()	indiapins (optional) Hardcoded city coords State centroids
Matching Engine	modules/matching.py (146 lines)	find_match(batch_doc_id)	geolocation.py SQLite (soil_deficiency, intake_batches)
Dosage Calculator	modules/dosage.py (98 lines)	get_dosage(compound, severity)	SQLite (dosage_rates)

5. Tamper-Evident Audit Chain Mechanism

Every pipeline execution generates a chronological chain of **hash-linked audit events** per batch. This is an **application-level tamper-evident mechanism** (not blockchain infrastructure). Each event's hash is computed from its own data plus the previous event's hash, forming an unbreakable chain.

Hash Formula:

```
SHA256( event_id | batch_id | event_type | timestamp | description | previous_hash )
```

Example Chain for BATCH-ZI2026-1014:

Event	Type	Actor	Previous Hash	Computed Hash
1	**BATCH_LOGGED**	Distributor Intake	GENESIS	SHA256(1|...)
2	**COMPLIANCE_PASSED**	Compliance Engine	hash(event 1)	SHA256(2|...)
3	**MATCH_CREATED**	Matching Engine	hash(event 2)	SHA256(3|...)
4	**DOSAGE_GENERATED**	Dosage Engine	hash(event 3)	SHA256(4|...)

Verification Process:

1. Fetch all audit_events for a batch, ordered by event_id ASC
2. For event 1: verify previous_event_hash == 'GENESIS'
3. Re-compute SHA256 hash using stored data fields
4. Compare computed hash with stored event_hash — mismatch = TAMPERING
5. For event N: verify previous_event_hash == event_hash of event (N-1)
6. If all events pass: '✓ All recorded audit chain events verified'

6. Rule-Based Dosage Calculation Logic

The platform uses a **Rule-Based Soil-Deficiency Dosage Adjustment Model**. It adjusts recommended base application rates according to deficiency severity tiers. This is a transparent, deterministic calculation — **not an AI/ML prediction**.

Compound	Application Type	Base Rate	Low (×1.25)	Medium (×1.0)	High (×0.75)
Zinc Sulphate (Zn)	Soil Application	37.5 kg/ha	46.88 kg/ha	37.50 kg/ha	28.13 kg/ha
Potassium Chloride (K)	Soil Application	30.0 kg/ha	37.50 kg/ha	30.00 kg/ha	22.50 kg/ha

Special Case — Ferrous Sulphate (Fe):

Ferrous Sulphate does **NOT** use soil kg/ha dosage. The base_rate_kg_ha is **NULL** in the database. Instead, it returns a **foliar spray specification** directly:

> **Foliar Guidance:** "3–4 sprays of 1.0% ferrous sulphate (FeSO■, 20% Fe) at weekly intervals during peak vegetative stage."

Severity Tier Interpretation:

• **Low** severity = higher deficiency in soil → **increased** dosage (+25%)
• **Medium** severity = standard deficiency → base rate as-is
• **High** severity = lower deficiency in soil → **reduced** dosage (–25%)

7. Soil-Deficiency Matching Strategy

The matching engine pairs accepted pharmaceutical batches with Indian states that have documented soil deficiency for the batch's specific compound. It uses a **two-priority strategy**:

Priority	Match Type	Logic	Distance
1 (Highest)	**Same-State Match**	Source state is 'usable' AND compound is listed in usable_compounds for that state	0 km (same state)
2 (Fallback)	**Nearest Fallback Match**	Scan ALL 29 states where status = 'usable'. Filter by compound compatibility. Compute Haversine great-circle distance from source to each candidate. Select nearest.	Haversine distance (km)
—	**No Match**	No usable state has the batch compound in its usable_compounds list. Pipeline stops.	—

Haversine Distance Formula:

The Haversine formula computes great-circle distance between two latitude/longitude coordinate pairs using Earth's radius (6,371 km). This ensures the nearest-fallback match is geographically optimal.

8. API Endpoint Map (24 Endpoints)

Method	Endpoint	Purpose
POST	`/api/process_batch/`	Execute 4-stage recovery pipeline (idempotent)
GET	`/api/batches`	List all intake batches
POST	`/api/batches`	Register a new batch
GET	`/api/batches/`	Fetch single batch details
POST	`/api/run_compliance`	Run compliance on all pending batches
GET	`/api/matches`	List all soil-deficiency matches
GET	`/api/matches/`	Single match detail with explanation
POST	`/api/matches//status`	Update match lifecycle status
POST	`/api/matches//assign-partner`	Assign certified partner facility
POST	`/api/matches//record-handoff`	Record material handoff completion
GET	`/api/partners`	List all partner facilities
GET	`/api/partners/`	Single partner detail
GET	`/api/partners/eligible`	Eligible partners with capacity checks
GET	`/api/audit/`	Fetch audit timeline for a batch
POST	`/api/audit//verify`	Verify SHA-256 chain integrity
GET	`/api/retests`	Soil re-test records & statistics
POST	`/api/retests`	Schedule a new soil re-test
POST	`/api/retests//result`	Submit post-application re-test result
GET	`/api/compounds`	Approved compound whitelist
GET	`/api/soil_deficiency`	29-state soil deficiency data
GET	`/api/stats`	Dashboard KPI statistics
GET	`/api/network_data`	Map data (sources, targets, partners, flows)
POST	`/api/simulate_recovery`	What-If dosage simulator
GET	`/api/reports/pdf/`	Download acknowledgement PDF (eligible only)

9. End-to-End User Workflow

Step	Page / Route	User Actions	System Response
1	■ **Dashboard /**	View platform overview	Displays KPIs: total batches, accepted count, matched count, recovery rate, compounds processed
2	■ **Stock & Compliance /stock**	Browse 45 intake records. Select a batch. Click 'Run Pipeline'.	Shows batch ledger with compound, quantity, source, expiry status. Navigates to /pipeline?batch=
3	■ **Recovery Pipeline /pipeline**	STEP 01: Select batch from dropdown. STEP 02: Click 'RUN RECOVERY PIPELINE'. STEP 03: View result. Click '■ Download PDF'.	7-step animated progress bar. Executes 4-stage pipeline. Displays match, dosage, partner. Generates acknowledgement PDF.
4	■ **Dosage Calculator /dosage**	Select compound, enter quantity, choose severity tier. Click 'Calculate'.	Shows adjusted rate, elemental nutrient yield, addressable hectares, calculation breakdown.
5	■■ **Recovery Map /soil-map**	Pan/zoom interactive map. Click colored markers. View flow lines.	29 element-color-coded markers. Popups with state deficiency profile. Partner pins and recovery flows.
6	■■ **About /about**	Read methodology & scope	Chemical standards, regulatory references, system design, scope boundaries.

Recovery Map — Element Color Legend:

■ **Green (#22c55e)**	Zinc (Zn) — Zinc Sulphate deficiency areas
■ **Blue (#2563eb)**	Iron (Fe) — Ferrous Sulphate deficiency areas
■ **Orange (#f97316)**	Potassium (K) — Potassium Chloride deficiency areas
■ **Gray (#6b7280)**	Non-Usable Region — no approved compound match

10. Project File Architecture

Path	Size	Description
SIH-193/		Repository root
■■■ README.md		Master project documentation
■		
■■■ starter_kit/starter/		Application directory
■■■ app.py	80 KB	Flask application & REST API (1981 lines)
■■■ app.db	240 KB	SQLite database (8 tables)
■■■ load_data.py	4 KB	Database initializer & seeder script
■■■ requirements.txt	—	Python dependencies
■		
■■■ modules/		Business Logic Modules
■ ■■■ intake.py	3 KB	Module 1: Whitelist & Compliance (106 lines)
■ ■■■ geolocation.py	10 KB	Module 4: Geolocation Resolution (223 lines)
■ ■■■ matching.py	6 KB	Module 2: Soil Matching Engine (146 lines)
■ ■■■ dosage.py	3 KB	Module 3: Rule-Based Dosage Calculator (98 lines)
■		
■■■ data/		Authoritative Reference Datasets
■ ■■■ compound_list.json	—	3 approved single-compound salts
■ ■■■ soil_deficiency.json	—	29 state-level deficiency records
■ ■■■ dosage_table.json	—	ICAR GRD base rates & adjustments
■ ■■■ demo_intake.csv	—	45 illustrative intake batch records
■ ■■■ districts.json	—	Illustrative district layer
■		
■■■ templates/		Jinja2 HTML Templates
■ ■■■ base.html	—	Master layout + sidebar navigation
■ ■■■ index.html	—	Dashboard
■ ■■■ stock.html	—	Stock & Compliance Ledger
■ ■■■ pipeline.html	—	Recovery Pipeline + PDF Download
■ ■■■ dosage.html	—	Dosage Calculator / Simulator
■ ■■■ soil_map.html	—	Recovery Map (Leaflet.js, 29 markers)
■ ■■■ about.html	—	Methodology & System Specs
■		
■■■ static/		CSS & JavaScript Assets
■ ■■■ style.css	—	Custom CSS design system
■ ■■■ script.js	—	Frontend interactivity
■		
■■■ tests/		Automated Test Suite (88 tests)
■■■ test_intake.py		Compliance validation tests
■■■ test_geolocation.py		Location resolution tests
■■■ test_matching.py		Soil matching tests
■■■ test_dosage.py		Dosage calculation tests
■■■ test_pipeline.py		End-to-end pipeline tests
■■■ test_final_restructure.py		Route, PDF, eligibility tests

■■■ `test_enhancements.py`		Enhancement feature tests
■■■ `test_phase2.py`		Phase 2 integration tests
■■■ `test_phase3.py`		Phase 3 match lifecycle tests
■■■ `test_phase4.py`		Phase 4 partner assignment tests
■■■ `test_phase5.py`		Phase 5 audit & re-test tests